

Lettura dei dati

GSTRUMSVIL - Guida allo sviluppo in BC - © SISTEMI S.p.A.

Ci siamo messi nelle condizioni di avere la struttura delle tabelle del nostro DB su Agis e ora possiamo utilizzarle per leggerne i dati.

Come abbiamo detto, in AGIS abbiamo la definizione della tabella (DFX), l'oggetto logico che ne descrive la struttura. Il dato vero e proprio è presente sul DB su Microsoft SQL Server.

Per accedere al dato possiamo utilizzare due modalità:

- tramite query;
- tramite oggetto.

Per entrambe, la connessione verso il DB viene fatta dall'ambiente di esecuzione, quindi non dobbiamo preoccuparci di aprire e chiudere la connessione durante le elaborazioni.

Se il programma viene eseguito da AGIS, la connessione viene aperta in quel momento e chiusa quando termina il programma. Quando si esegue da AGIS (o da BCDevelop) nei parametri di esecuzione è indispensabile andare ad indicare l'AdL sul quale andare a reperire i dati relativi al DB che si vuole utilizzare.

Interazione con il DataBase tramite Query

L'esecuzione di un oggetto query può esser fatto utilizzando il datareader.

Indipendentemente dal numero di record estratto dalla query, dovremo utilizzare un ciclo *WHILE* per scorrere gli elementi. Se la nostra query dovesse estrarre un solo record, allora il ciclo avverrà una volta sola.

Vediamo il datareader nel dettaglio.

Lettura di un set di record

Il datareader permette di leggere il contenuto dell'oggetto query dichiarato e definito in precedenza. Vediamo come.

Prendiamo una query che legga tutta la tabella degli articoli:

SQL**SELECT**

CodArt AS [CodArt]
,DesArt AS [DesArt]

FROM BLDArticoli AS [BLDArticoli]**WHERE** (DBGruppo = 'GG')

Il sorgente di definizione della query è il seguente:

BC - BLDOBJQRY_QUERYBASE_DATAREADER_DEFINIZIONE

```
'@PGQUERY BLDOBJQRY_QUERYBASE_DATAREADER_DEFINIZIONE STRICT[4] ' Esercitazione
query per DataReader - Definizione query

'@SBP QueryObject[BCQUERY] [INOUT]

' Definizione dell'identificativo della query
'@DEFQUERY QUERY[QueryObject] NOME["BLDOBJQRY_QUERYBASE_DATAREADER_DEFINIZIONE"]

' Definizione della tabella principale (FROM)
'@FROM TABELLA[BLDART]

' Definizione elenco dei campi da estrarre
'@COLUMN ESPR[BLDART.COD AS [CodArt]] _
          ESPR[BLDART.DES AS [DesArt]]

'@ENDQUERY

End
```

Ora proviamo a leggerli in sequenza ed emettere un messaggio per ogni articolo.

Per poter utilizzare il datareader è necessario dichiarare la variabile con il nuovo tipo DATAREADER. A tutti gli effetti questa variabile è un oggetto al quale sono associati i metodi per la lettura dei dati:

Funzione	Descrizione
Next()	Al primo richiamo esegue il controllo se il recorset è vuoto, in caso affermativo torna #False , altrimenti #True Nei richiami successivi, avanza il puntatore Controlla il raggiungimento della fine del recordset.
Rewind()	Ripete la lettura dall'inizio
Exists()	Verifica se il datareader ha dati da leggere
First()	Legge il primo record della query

Vediamo il richiamo nel sorgente BC:

BC - SUB76_QUERYBASE_DATAREADER

```
'@PG SUB76_QUERYBASE_DATAREADER STRICT[4] ' Esercitazione 76: Query base con
datareader

'@SBP

DIM queryObject[DATAREADER[BLDOBJQRY_QUERYBASE_GETQUERYDATA()]]

DIM messaggio[STRING]
DIM contatore[INT]

While queryObject.Next()

    contatore += 1
    messaggio = FNFORMAT$("Codice: {0} Descrizione: {1} Articolo letto:
{2}", queryObject.CodArt, queryObject.DesArt, contatore)
    MESSAGEBOX(#INFO, "CBAS_76", messaggio )

EndWhile

End
```

Per richiamare il nostro oggetto query creato in precedenza abbiamo due modi:

- una forma contratta: permette di dichiarare e definire l'oggetto query in unico punto. È da preferirsi quando l'utilizzo è puntuale e si hanno a disposizione tutti gli eventuali parametri che servono la per la sua definizione
- una forma esplicita: la dichiarazione e la definizione avvengono in due punti differenti del programma. Da utilizzarsi nel caso in cui serva definire la query in più punti oppure non si hanno disposizione tutti i suoi parametri

La forma contratta è visibile nell'esempio sopra riportato:

```
[SHARED] DIM <NomeVar>[DATAREADER[<Nome tipo oggetto 47>(par1, par1, ecc)]]
```

dove DATAREADER è la keyword che ci permette di definire la variabile come oggetto datareader.

La forma esplicita prevede la dichiarazione e la definizione in due punti differenti:

```
[SHARED] DIM <NomeVar>[DATAREADER[<Nome tipo oggetto 47]]
```

```
...
```

```
<NomeVar>.Definizione(par1, par2, ecc)
```

```
...
```

La dichiarazione non prevede più i parametri, perché sarà cura del metodo Definizione andare ad utilizzarli.

Il ciclo while viene incrementato dal metodo Next() e al suo interno possiamo accedere alle proprietà dell'oggetto query per poter visualizzare il contenuto tramite la MESSAGEBOX.

Il contenuto delle singole proprietà viene popolato dall'ambiente con i valori letti da DB. La query viene eseguita al primo accesso del metodo, in questo caso il Next().

Nel filtro vediamo comparire un campo in più, il DBGruppo, questo perché la tabella BLDART è di gruppo. Nella definizione del filtro della query in BC non è necessario andare ad indicarlo, a meno che non si debba andare a leggere un gruppo diverso da quello corrente. L'ambiente di esecuzione lo va ad impostare in automatico quando elabora la query.

Nel caso avessimo bisogno di leggere il primo record possiamo utilizzare il metodo First() in modo da evitare il ciclo While. Questo metodo può essere utilizzato anche nel caso in cui sappiamo a priori che la query possa estrarre un record solo.

Interazione con il Database tramite Classi

Fino ad ora abbiamo visto come viene definito e trattato un oggetto, senza fare alcun riferimento a come questo viene popolato a partire dal database.

Nel caso in cui il DFX è anche la descrizione di una tabella fisicamente presente sul database, la classe potrà essere utilizzata per istanziare oggetti che possono interagire con la base dati.

Partiamo considerando la classe con DFX di riferimento e senza altre proprietà. Questa, come abbiamo visto in precedenza, è strettamente collegata ad un DFX e di conseguenza ad una tabella del database.

Il metodo base .LoadFromDB() consente il caricamento dei dati dal database. Per poterlo usare è necessario aver impostato sull'oggetto la chiave primaria della tabella. Ipotizzando di dover caricare i dati della classe BLDCLIFOR vanno eseguite le seguenti operazioni:

BC

```
DIM CodCliFor[INT] = 5
DIM varCliente[CLASSE[BLDCLIFOR]]

' Assegna la chiave primaria della tabella client/fornitori
varCliente.COD = CodCliFor

' Esegue il caricamento dei dati
varCliente.LoadFromDB()
```

Questa operazione consente il reperimento del record, sulla tabella di riferimento del DFX, e popola le proprietà dell'oggetto cliente.

Questa modalità lavora solo con la chiave primaria, quindi senza filtri aggiuntivi.

Il caricamento della classe dal database, oltre ai dati, correda l'oggetto di ulteriori informazioni utili allo sviluppo e, come vedremo in seguito, utili anche alla fase di aggiornamento dei dati.

In memoria infatti viene mantenuta una copia dei dati originali caricati dal DB. Questo vuol dire che, dopo aver modificato le proprietà della classe, è possibile accedere ai dati originali e sfruttarli per eseguire operazioni senza dover tornare a leggere da database.

Inoltre l'oggetto possiede una proprietà aggiuntiva che contiene lo stato in cui esso si trova.

I valori che può assumere sono:

Valore	Stato	Descrizione
0	Nuovo	Oggetto vuoto o pulito tramite CLEAR()
1	Originale	Oggetto caricato da database
2	Modificato	Oggetto caricato da database e successivamente modificato
3	Nuovo Modificato	Oggetto non letto da database successivamente modificato

Per capire se l'oggetto ha subito modifiche dopo il suo caricamento da database va usato il metodo `.IsModified()`. Questo può essere usato a livello di oggetto o di singola proprietà.

Per identificare invece se un oggetto è nuovo e non deriva da DB va usato il metodo `.IsNew()`.

Tramite il metodo base `.GetOriginal()` otteniamo invece i dati originali restituiti in un oggetto dello stesso tipo.

 Il metodo `.IsModified()` va utilizzato solo se l'oggetto è stato caricato da database. Su oggetti che non sono stati caricati da database non sono presenti i dati originali per cui il metodo restituisce 1 se l'oggetto è in stato Nuovo Modificato.

Un esempio di utilizzo dei dati originali è la gestione dell'azione per annullare le modifiche fatte in una videata. Aspetto che approfondiremo quando parleremo di utilizzo di oggetti e collezioni per l'esposizione dei dati a video. Ma lo vedremo più avanti.

Se invece dovessimo caricare una collezione di oggetti riferiti ad una classe occorrerebbe necessariamente utilizzare il metodo `ToCollection()` del `DATAREADER`.

 Nel caso di utilizzo di una query il binding tra colonne della query e proprietà dell'oggetto viene fatto per nome. Le colonne della query dovranno quindi avere come alias il nome della proprietà.

Ogni singolo oggetto della collezione ha il suo specifico stato rispetto al momento in cui è stato caricato da database. È possibile anche utilizzare il metodo `.IsModified()` direttamente sulla collezione per sapere se almeno un oggetto ha subito modifiche.

Inoltre eventuali oggetti rimossi dalla collezione vengono mantenuti in memoria in una collezione parallela.

Questo significa che anche in questo caso possiamo accedere a questi dati per capire quali oggetti sono stati eliminati. Per farlo si utilizza il metodo `.GetDeleted()`. Quest'ultimo restituisce una collezione degli oggetti eliminati successivamente al caricamento dei dati dal database.

Dopo aver fatto delle modifiche ad un oggetto/collezione è possibile ripristinare i valori letti da DB, annullando quindi le modifiche fatte in memoria, tramite il metodo `.Restore()`.

Caricamento di una classe composta

Nel paragrafo in cui è stata esposta la definizione di una classe abbiamo volutamente sorvolato alcuni aspetti che influiscono nei meccanismi di interazione con il database.

Abbiamo visto come una classe possa essere costituita da una composizione, anche gerarchica, di altre classi. Come si carica una classe composta? Per effettuare il caricamento dei dati da database si procede come visto in precedenza, ovvero impostando la chiave sull'oggetto e poi eseguendo il metodo di `.LoadFromDB()`. In questo caso va fatto riferimento alla chiave della tabella di associata alla classe (tramite il DFX di riferimento o tramite la classe impostata come proprietà `_BASE`).

Riprendendo la definizione della classe `BLD_CLIENTE` vista in precedenza, la sintassi per il caricamento dei dati è analoga a quella vista per la classe semplice `BLDCLIFOR`:

BC

```
DIM CodCliFor[INT] = 5
DIM tmpCliente[CLASSE[BLD_CLIENTE]]

' Assegna la chiave primaria della tabella client/fornitori
tmpCliente.COD = CodCliFor

' Esegue il caricamento dei dati
tmpCliente.LoadFromDB()
```

La tabella di riferimento non ha il solo scopo di definire la chiave di caricamento dei dati da database.

Questa definisce anche il punto di partenza per reperire i legami con le altre tabelle che definiscono la gerarchia della classe. Diventano quindi essenziali le relazioni tra le tabelle per sfruttare a fondo le potenzialità dell'ambiente Sistemi.

Nella definizione della classe BLD_CLIENTE viene chiesto quale tipo di collegamento c'è tra la classe principale e gli indirizzi. Nel caso venga indicata la mappatura con il DB, allora è necessario indicare la relazione che intercorre tra le due tabelle:

Num...	Nome	Nome esteso	Classe	Mappatura con il Database	Relazione	Lung.	Num. Cifre	Num. Dec.	Elem. Schiera	Default	Valori ammessi	Obi
24,00	NSEDE	CodIndSedeLegale					4	5	0	0	Tutti	
25,00	NSPED	CodIndSpedizione					4	5	0	0	Tutti	
26,00	NFATT	CodIndFatturazione					4	5	0	0	Tutti	
27,00	CDAT	CreazioneData					4	0	0	0	Tutti	
28,00	CORA	CreazioneOra					4	6	0	0	Tutti	
29,00	CWS	CreazioneStazione					2	0	0	0	Tutti	
30,00	COPE	CreazioneOpe					20	0	0	0	Tutti	
31,00	VDAT	VariazioneData					4	0	0	0	Tutti	
32,00	VORA	VariazioneOra					4	6	0	0	Tutti	
33,00	VWS	VariazioneStazione					2	0	0	0	Tutti	
34,00	VOPE	VariazioneOpe					20	0	0	0	Tutti	
35,00	Indirizzi	Indirizzi	BLD_INDIRIZZI	Si	INDCLIFOR_BLDCLIFOR		0	0	0	0	Tutti	

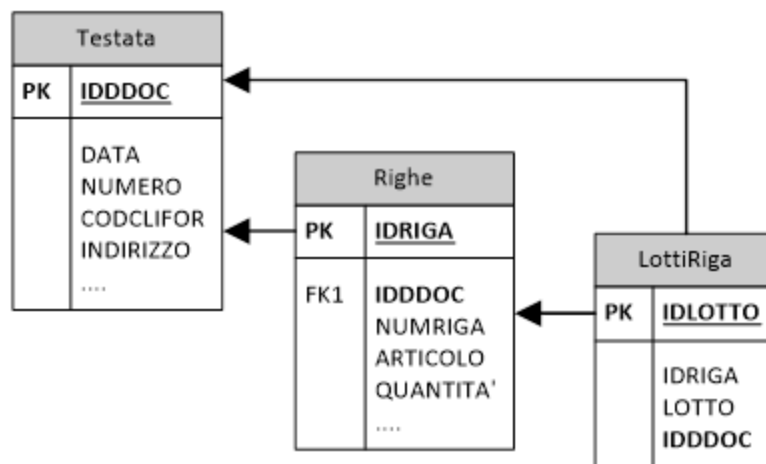
Ci ritorneremo sopra più avanti.

Riprendiamo sempre la definizione della classe BLD_CLIENTE, al suo interno è definita anche una collezione di indirizzi. Questa fa riferimento al descrittore BLDINDCLIFOR. Essendo i due descrittori riferiti a due tabelle tra loro dipendenti (non possono esserci indirizzi senza un cliente), è stata definita una relazione tra i dfx BLDINDCLIFOR e BLDCLIFOR sul campo CCF (codice cliente/fornitore).

L'esistenza della relazione fa in modo che il metodo LOADFROMDB() possa caricare in automatico la collezione degli indirizzi riferiti al cliente per cui è stato richiesto il caricamento dei dati da database.

Dietro le quinte vengono quindi analizzate le relazioni a partire dal DFX riferito dalla classe (o dalla proprietà _BASE) e propagata la chiave primaria della tabella per eseguire il caricamento dei dati delle tabelle relazionate.

Ipotizziamo quindi di avere la seguente struttura di tabelle :



Il caricamento di una Classe che contiene la struttura gerarchica riportata sopra viene pilotata dall'impostazione della chiave primaria della tabella base, ovvero il campo IDDDOC della classe di riferimento della tabella testata. Avendo creato una relazione con la tabella delle righe viene automaticamente innescata la lettura dati da database delle righe filtrando sul campo IDDDOC. La stessa cosa avviene sulla tabella LottiRiga che prevede una relazione con la tabella Righe.

Nell'esempio sopra riportato in tutte le tabelle è stato indicato il campo IDDOC, è comunque possibile avere sulla tabella principale (Testata) il campo chiave ID e nelle tabelle in relazione definire i campi nelle foreign key con nome diverso (es: IDDOC), l'importante è che nelle tabelle secondarie il campo abbia lo stesso nome.

⚠ Quando viene richiesto il caricamento di più livelli, la lettura dei dati viene eseguita propagando il valore della chiave ai livelli più bassi, utilizzando lo stesso nome (o il nome in relazione) dei campi chiave nella foreign key.

Questo viene fatto per ottimizzare le operazioni ed eseguire una sola SELECT per ogni tabella.

La relazione, che potrebbe non comprendere il campo chiave o includere altri campi/valori costanti, viene invece usata per associare in memoria i record letti.

Se il campo chiave della tabella principale non è presente sui sottolivelli, allora il caricamento avviene in modo non ottimizzato sfruttando la relazione e facendo più SELECT.

Ad esempio, se nelle tabelle riportate sopra mancasse il campo IDDDOC sulla tabella LottiRiga, per caricare i lotti verrebbe utilizzato il campo IDRIGA e quindi eseguita una SELECT per ogni riga.

Il caricamento della classe può essere completo oppure può essere eseguito fino ad un certo livello. Il metodo `.LoadFromDB()` prevede infatti un parametro per indicare il livello di profondità con cui caricare i dati. Se omesso viene letta l'intera struttura, se indicato zero viene letta solo la tabella di riferimento (o quella della proprietà `_BASE`) altrimenti può essere definito uno specifico livello a cui la lettura dei dati si deve fermare.

La restante parte dei dati che compongono la classe viene letta quando serve tramite un meccanismo di lettura automatico chiamato lazy loading.

Lazy loading

Ipotizziamo di dover rappresentare a video la struttura dati descritta precedentemente. Molto probabilmente prima che l'utente arrivi a visualizzare il dettaglio di una singola riga con i suoi lotti dovrà passare da diverse videate.

Utilizzando il livello massimo di lettura dei dati si può evitare di caricare preventivamente da Database anche tutti i lotti di tutte le righe. Per farlo indicheremo il livello 1 nel metodo `.LoadFromDB()` utilizzato per caricare i dati nel nostro oggetto composto che rappresenta il documento, ottenendo quindi il caricamento della testata, delle sue righe ma non dei lotti.

Quest'ultimi saranno caricati solo nel momento in cui si farà effettivamente accesso all'oggetto lotto di una riga. Quindi se il dettaglio riga presenta i lotti ad essa associati questi saranno letti dal database nel momento in cui si accede al dettaglio della riga.

I dati vengono caricati sempre sfruttando le relazioni, quindi è necessario che la tabella `LottiRiga` sia relazionata alla tabella `Righe`. Questo meccanismo è molto potente e consente di eseguire il caricamento dei dati solo quando vengono messi in gioco. Tecnica utilizzata anche in modo molto efficace per agevolare le personalizzazioni, capitolo che vedremo più avanti.

Ipotizzando quindi il richiamo del metodo carica del documento nel seguente modo:

```
BC

' Richiamo metodo con livello di caricamento 1
' (Identificativo documento, Livello di caricamento)
Fattura.Carica(IDD_DOC, 1)
```

Le operazioni eseguite sul Database saranno la `SELECT` per il caricamento della testata e quella per il caricamento di tutte le righe.

Dopo questa istruzione quindi i lotti non sono presenti nell'oggetto.

Per innescare il caricamento dei lotti di una riga non servono operazioni esplicite, ma essendo un componente relazionato alle righe, un qualsiasi accesso alla collezione dei lotti di una riga ne determina la lettura dei dati da Database.

Ad esempio un ciclo `.ForEach()` per l'esposizione in griglia:

```
BC
```

```
' Richiamo metodo con livello di caricamento 1
' (Identificativo documento, Livello di caricamento)
Fattura.Riga.Item(1).Lotti.ForEach(tmpLotti, CaricaRigaLottiInGriglia(tmpLotti
[INOUT]))
```

Non appena il programma accede alla proprietà dei LOTTI, se su tale collezione non è ancora stato eseguito in precedenza il caricamento dei dati dal Database, questo avviene in automatico in modo preventivo. Da questo momento in avanti nell'oggetto composto sono presenti solo i lotti della prima riga. Allo stesso modo accedendo ad altre righe del documento saranno caricati in modo automatico i lotti dal Database.

Il caricamento in lazy loading dei dati presuppone l'utilizzo del metodo base `.LoadFromDB()` per il caricamento automatico dei componenti di un composto.

Nel caso in cui il componente preveda invece un metodo standard in sostituzione del metodo base `.LoadFromDB()` questo sarà richiamato dall'ambiente di esecuzione garantendo la logica di caricamento prevista per esso. Facendo riferimento all'esempio riportato sopra, se la classe di riferimento dei lotti avesse il metodo `.Carica()` definito in sostituzione del metodo base, questo sarebbe stato richiamato per ogni lotto collegato alla singola riga del documento prima di eseguire il metodo `.ForEach()`.



È possibile disinnescare il meccanismo di Lazy loading tramite il metodo `.EnableLazyLoading()`. Vedremo in seguito in quali contesti può essere applicato.

Mappatura con il database

All'interno di una classe è possibile, per le proprietà di tipo classe o collezione, definire come queste si devono comportare quando vengono eseguiti i metodi di base per operare sul database. In particolare nel campo "Mappatura con il database" è possibile indicare uno dei seguenti valori:

Valore	Comportamento
Si	La proprietà viene caricata dal Database ed eventuali modifiche vengono concretizzate sul database. Va specificata la relazione da utilizzare per il caricamento dei dati. La stessa viene utilizzata anche per l'operazione di delete in cascata.
Solo lettura automatica	La proprietà viene caricata dal Database al primo utilizzo tramite il meccanismo di Lazy loading. Va specificata la relazione da utilizzare per il caricamento dei dati. Eventuali modifiche non vengono concretizzate sul database.

Valore	Comportamento
Non relazionata	La proprietà viene ignorata sia in fase di caricamento dati da database che in fase di aggiornamento. E' cura del programmatore valorizzarla ed eventuali modifiche non vengono concretizzate sul database.

Le proprietà per cui viene impostato il valore "Solo lettura automatica" sono caratterizzate dall'essere escluse dall'aggiornamento dei dati sul database. Sono informazioni aggiuntive, utili alla gestione dell'oggetto, che pur facendo riferimento ad una tabella non sono rilevanti ai fini della scrittura. Queste sono anche ignorate dalla fase di caricamento dei dati dal database tramite il metodo `base.LoadFromDB()`. Il loro popolamento è demandato al loro effettivo utilizzo, tramite il meccanismo di Lazy Loading descritto in precedenza.

Un esempio pratico può essere l'utilizzo della classe `BLDCLIFOR` come decodifica all'interno della classe documento fattura:

AGOGG - BLD_FATTURA - Gestione Classe composta (Prodotto AGS: BLD_DEMO/BLD_DEMO)

Classe: **BLD_FATTURA**

Posizione a campo:

Numero	Base	Nome	Descrizione	Visibilità	Tipo	Classe	Mappatura con il Database	Relazione	Lung.	Num. Cifre	Num. Dec.
1,00	☒	_BASE	Testata	Publico	#	BLDTESTATA			0	0	0
2,00	☐	RIGA	Righe della testata	Publico	%	BLD_RIGA	SI	BLDRIGA_BLDTESTAT	0	0	0
3,00	☐	PIEDE	Piede della testata	Publico	#	BLDPIEDE	SI	BLDPIEDE_BLDTESTAT	0	0	0
4,00	☐	CLIENTE	Cliente della testata	Publico	#	BLDCLIFOR	Solo lettura automatica	BLDTESTATA_BLDCLIFOR	0	0	0
5,00	☐	INDIRIZZO	Indirizzo della testata	Publico	#	BLDINDCLIFOR	Solo lettura automatica	BLDTESTATA_BLDIND	0	0	0
6,00	☐	MAGAZZINO	Magazzino	Publico	#	BLDMAGAZZINO	Solo lettura automatica	BLDTESTATA_BLDMAI	0	0	0
7,00	☐	MAGAZZINOS	Magazzino secondario	Publico	#	BLDMAGAZZINOS	Solo lettura automatica	BLDTESTATA_BLDMAI	0	0	0
8,00	☐	COLLOCAZIONE	Collocazione mag	Publico	#	BLDCOLLOCAZIONE	Solo lettura automatica	BLDTESTATA_BLDCCI	0	0	0
9,00	☐	COLLOCAZIONES	Collocazione mag secondario	Publico	#	BLDCOLLOCAZIONES	Solo lettura automatica	BLDTESTATA_BLDCCI	0	0	0
10,00	☐	COND_PAG	Condizione di pagamento	Publico	#	BLDCPG	Solo lettura automatica	BLDTESTATA_BLDCCP	0	0	0
11,00	☐	SCADENZA	Scadenza di pagamento	Publico	#	BLDSDE	Solo lettura automatica	BLDTESTATA_BLDSCD	0	0	0
12,00	☐	DESAUTO	Automezzo: descrizione	Publico	C				35	0	0
13,00	☐	ORA_FAT	Ora	Publico	L				4	4	0

Abbbandona (F1) Copia campi (F3) Valori campo (F8) Query di caricamento (F11) Visualizza Classe (F12) Salva (End)

PETRASSO GG/01/01

In questo caso quando viene eseguito il metodo `.LoadFromDB()` per caricare l'oggetto fattura da database, le proprietà che prevedono "Solo lettura automatica" vengono ignorate. Queste vengono caricate quando avviene il primo utilizzo, ad esempio quando si espone a video la ragione sociale del cliente intestatario della fattura.

L'oggetto cliente all'interno della fattura, dopo il primo caricamento avvenuto con il lazy loading, va aggiornato rispetto al codice cliente impostato sulla fattura. Sarà quindi la parte di codice che esegue la validazione della proprietà codice cliente indicato in testata che si dovrà preoccupare di aggiornare l'oggetto eseguendo il relativo metodo carica del cliente.

⚠ Per eseguire un nuovo caricamento da database dei dati è necessario eseguire il metodo `.Clear()` sull'oggetto prima di eseguire il metodo `.LoadFromDB()`. Va inoltre considerato che se le classi utilizzate come decodifica hanno un metodo di caricamento specifico, utilizzato per popolare dati aggiuntivi, questo non viene normalmente utilizzato, tranne nel caso in cui un metodo carichi in override (Sostituzione

metodo base). Il meccanismo di lazy loading, salvo casi di metodi che caricano in override, esegue il caricamento base dei dati eseguendo la semplice SELECT per caricare i dati dalla tabella indicata come _BASE.

Le proprietà scollegate dalla proprietà impostata come base vanno invece marcate con il valore "Non mappato". Per queste non verrà eseguito il caricamento dei dati e saranno ignorate dalla fase di aggiornamento dati su database. Vanno quindi considerati dati di servizio caricati a cura del programmatore.

Caricamento manuale

In alcuni contesti il metodo automatico di caricamento dei dati può risultare il metodo meno efficiente in termini di operazioni su database in special modo se l'oggetto deve essere corredato da dati aggiuntivi non presenti sul database.

Uno scenario di possibile utilizzo del caricamento manuale della classe è rappresentato dall'esposizione in griglia di dati di decodifica. Ipotizziamo infatti di dover esporre in griglia le righe del documento e che la classe di riferimento contenga la classe di decodifica dell'articolo. Come descritto in precedenza, l'oggetto articolo della singola riga viene caricato da database solo quando lo si utilizza. Questo significa che nel momento in cui si esegue il caricamento della griglia, e si espone una proprietà dell'articolo, ad ogni inserimento di una riga nella griglia verrà scatenata la lettura dell'articolo di riferimento della riga. Ne consegue che se la griglia ha molte righe, e magari l'articolo non è neanche l'unica decodifica, si dovranno eseguire molte operazioni su database per popolare la griglia con probabili problemi di performance.

Un modo alternativo è il caricamento manuale. Rimaniamo sull'esempio descritto sopra e proviamo a vedere il codice che servirebbe scrivere per caricare la classe in modo manuale.

BC

```
DIM Fattura[CLASSE[BLD_FATTURA]]

' Assegna la chiave primaria della tabella BLDTESTAT
Fattura.IDD = IDD_SEL

' Esegue il caricamento dei dati del solo livello _BASE
Fattura.LoadFromDB(0)
```

A questo punto abbiamo caricato da database i dati della testata.

Ora vediamo come caricare la collezione di righe e classe di decodifica dell'articolo e la sua assegnazione alle righe di riferimento.

BC

```

' Definizione collezione articoli
SHARED DIM ElencoArticoli[CLASSE[BLDARTICOLO] COLLEZIONE]

' Lettura degli articoli (tramite query)
SHARED DIM QryArticoli[QUERY[QueryObject]
'@GETQUERYDATA QUERY[QueryObject] COLLEZIONE[ElencoArticoli]

' Lettura collezione
DIM tmpRiga[CLASSE[BLD_RIGA]]
Fattura.Riga.ForEach(tmpRiga, AssegnaArticolo(tmpRiga [INOUT]))

'@SRP AssegnaArticolo(singRiga[CLASSE[BLD_RIGA]] [INOUT])

' Disattivo il meccanismo di Lazyloading dei dati per i componenti dell'oggetto
riga
singRiga.EnableLazyLoading(0)

' Cerca l'articolo a cui fa riferimento la riga e lo assegna alla riga stessa
singRiga.Articolo => ElencoArticoli.FindFirst(BLDARTICOLO.CART = singRiga.CART)

' Riattivo il meccanismo di Lazyloading dei dati per i componenti dell'oggetto
singRiga.EnableLazyLoading(1)

Return

```

Da notare che le righe vengono caricate già al momento dell'esecuzione del metodo .ForEach().

Una volta eseguita l'assegnazione a tutte le righe del rispettivo articolo la griglia di elenco delle righe del documento può essere caricata senza accessi al database esponendo anche proprietà dell'articolo.

⚠ L'utilizzo del metodo .EnableLazyLoading() è fondamentale in questo tipo di caricamento. Se non utilizzato per disattivare il caricamento automatico l'esecuzione dell'assegnazione dell'oggetto ARTICOLO sulla riga innescherebbe la lettura sul database rendendo vano il tentativo di ottimizzare gli accessi al database.

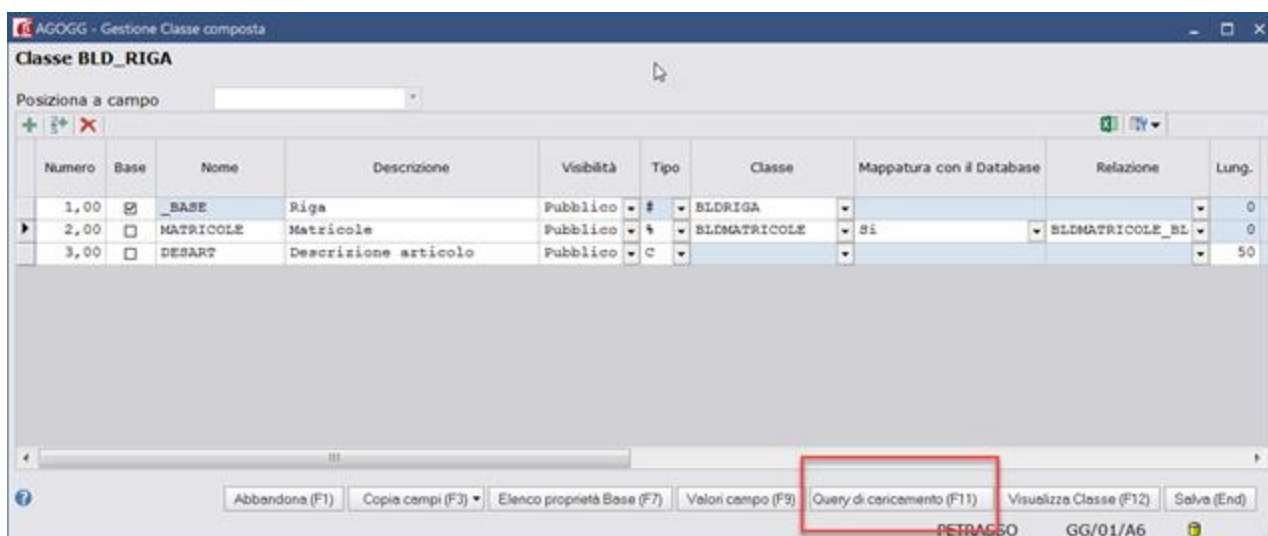
💡 Per semplificare l'esempio la collezione di articoli è stata caricata senza alcun filtro, portando quindi in memoria l'intero contenuto della tabella articoli. Questa operazione può essere ottimizzata definendo una query per estrarre i soli articoli interessati dal documento oggetto di caricamento. La collezione ElencoArticoli verrebbe quindi caricata tramite il metodo ToCollection() del DATAREADER.

Query di caricamento dati

Un altro scenario che potremmo doverci trovare a gestire è il caricamento di una classe con dati aggiuntivi rispetto alla definizione della base dati.

Riprendiamo l'esempio precedente ma, a differenza di prima, ipotizziamo di non aver utilizzato la classe di decodifica dell'articolo e di aver portato parte delle informazioni dell'articolo direttamente come proprietà aggiuntive della classe che rappresenta una riga.

Ipotizziamo quindi di avere la classe definita così:



Il caricamento automatico dei dati non lavora sulle proprietà definite in modo diretto sulla classe, senza aver quindi un riferimento ad un'altra classe e quindi ad una tabella. Per cui in questi casi non è possibile sfruttare gli automatismi descritti in precedenza senza scrivere del codice che valorizzi queste proprietà di servizio. In questo caso la proprietà DESART non sarebbe quindi valorizzata dopo l'esecuzione del metodo `.LoadFromDB()`.

Un modo semplice per farlo è quello di completare il caricamento dei dati leggendo la collezione di righe, caricata preventivamente, e valorizzando per ogni riga le proprietà di servizio reperendo le informazioni dal database. Rispetto al precedente esempio possiamo quindi dire che al posto di assegnare l'oggetto di decodifica articolo verranno assegnate le singole proprietà della riga:

BC

```

' Definizione collezione articoli
SHARED DIM ElencoArticoli[CLASSE[BLDARTICOLO] COLLEZIONE]

' Lettura degli articoli (tramite oggetto query)
'@GETQUERYDATA QUERY[QueryObject] COLLEZIONE[ElencoArticoli]

' Lettura collezione
DIM tmpRiga[CLASSE[BLD_RIGA]]
Fattura.Riga.ForEach(tmpRiga, AssegnaProprietaAggiuntive(tmpRiga [INOUT]))

'@SRP AssegnaProprietaAggiuntive(singRiga[CLASSE[BLD_RIGA]] [INOUT])

' Cerca l'articolo a cui fa riferimento la riga e lo assegna alla riga stessa
singRiga.DESART = ElencoArticoli.FindFirst(BLDARTICOLO.CART =
singRiga.CART).DESART

Return

```

La logica è molto simile a quella vista prima, la differenza sta nel fatto che sulla riga non avrò a disposizione tutte le proprietà dell'articolo ma solo quelle previste nella classe che rappresenta la riga.

Con questa impostazione un metodo alternativo, e più ottimizzato rispetto alle letture su database, è l'utilizzo della query di caricamento.

Ad ogni classe può infatti essere associata la definizione di una query per il caricamento dei dati. L'associazione avviene utilizzando la specifica funzione presente in fase di definizione della classe evidenziata nell'immagine precedente.

La query di caricamento è associata alla classe tramite la creazione automatica di un nuovo oggetto query.

Se la classe prevede la query di caricamento dati, questa verrà eseguita in sostituzione della normale SELECT prevista dal metodo .LoadFromDB(). In questo modo si può passare dall'utilizzo del metodo base di caricamento dati ampliando l'elenco delle colonne rispetto a quelle previste dalla tabella di riferimento della classe. Riprendendo lo scenario descritto in precedenza potremmo quindi definire una query che prevede la tabella BLDARTICOLI in JOIN con la tabella BLDRIGHE ed estrarre la descrizione dell'articolo da popolare su ogni singola riga nella proprietà specifica. Definendola il caricamento della classe può avvenire semplicemente richiamando il metodo base .LoadFromDB(), la query verrà utilizzata in automatico dall'ambiente di esecuzione al momento del caricamento.

 Il binding tra colonne della query e proprietà della classe avviene per nome, la query dovrà quindi essere definita avendo cura di indicare per ogni colonna l'alias uguale al nome della proprietà che dovrà contenere tale dato.

La Definizione della Query non può prevedere parametri dinamici da applicare come filtri, l'unico filtro applicato è la chiave della tabella di riferimento applicato in automatico dall'ambiente a run time.



La query di caricamento viene utilizzata per il caricamento dei dati a tutti i livelli di composizione della classe. Per ogni livello vengono caricati il `_BASE` e le proprietà aggiuntive.

Cache dati letti da Database

Abbiamo visto come è possibile caricare un oggetto leggendo i dati dal database e gli elementi a corredo che vengono resi disponibili. Per quanto abbiamo visto ogni volta che viene eseguito il metodo `.LoadFromDB()`, oppure quando viene innescato il lazy loading, viene eseguita un'operazione sul database per reperire i dati. Nel modello di programmazione OOP è anche possibile attivare un meccanismo di cache per evitare di ripetere le operazioni di lettura dei dati già eseguite. Vediamo come funziona.

Lo scopo della cache, anche nel modello OOP, è quello di evitare di tornare a reperire gli stessi dati dal Database. Per cui ogni cache fa riferimento ad una singola tabella. Questo vuol dire che la cache è strettamente collegata ad una classe che ha un DFX di riferimento da cui derivare la chiave di identificazione del singolo record.

L'attivazione della cache va quindi effettuata a livello di singola classe tramite il metodo statico `.EnableCache()`:

BC

```
' Attivazione cache per la classe BLDCLIFOR  
BLDCLIFOR.EnableCache()
```

L'attivazione può essere eseguita una volta sola per tutte le classi per cui si vuole attivare il meccanismo di cache della lettura dati. Tipicamente verrà quindi eseguita nel modulo della gestione. Da considerare che questo è un metodo statico, ovvero riferisce alla classe e non al singolo oggetto istanziato. Di conseguenza tutti gli oggetti di tipo `BLDCLIFOR` sfrutteranno la cache.

Una volta attivata la cache l'esecuzione del metodo `.LoadFromDB()`, prima di eseguire l'operazione di lettura dei dati dal database, controlla la presenza in cache della specifica chiave richiesta. Se presente popola l'oggetto con i dati presenti in cache senza tornare a leggere da database. Se non presente legge i dati dal database e li inserisce in cache per le eventuali letture che saranno eseguite successivamente sulla stessa classe.

Anche in caso di oggetto non trovato, il record vuoto viene inserito in cache in modo tale da evitare successive riletture sul DB.

Come detto in precedenza la cache è gestita solo per le classi semplici in quanto identificano la tabella fisica da cui reperire i dati. E nel caso di classi composte? La composizione di classi prevede come tasselli di base delle classi semplici. Di conseguenza anche il lazy loading può sfruttare la cache se

opportunamente attivata. Quello che è importante considerare è che non viene gestita la cache dell'oggetto composto ma dei suoi singoli elementi base che riferiscono a classi semplici.

La cache viene popolata e letta anche nel caso di utilizzo di proprietà di tipo collezione all'interno di altre classi.

Oltre al metodo `.EnableCache()` per l'attivazione è presente il metodo `.ClearCache()` per poter forzare lo svuotamento dei dati presenti in cache:

BC

```
' Elimino i dati presenti in cache  
BLDCLIFOR.ClearCache()
```

Questo metodo può tornare utile nel momento in cui è necessario fare in modo che il programma torni a leggere i dati dal database. Ad esempio potremmo voler tenere in cache i dati del cliente per tutta la fase di gestione di un documento e svuotarla nel momento in cui si memorizzano i dati per fare in modo che la gestione del successivo documento torni a rileggere i dati dal database.

Il metodo `.ClearCache()` non è l'unico modo per invalidare la cache. Infatti una qualsiasi operazione di scrittura sulla tabella collegata alla cache invalida automaticamente la cache, ovvero la svuota implicitamente.

Un'altra possibilità è quella di inibire l'utilizzo della cache per il singolo oggetto. Questo tipo di impostazione può essere utile nel momento in cui si vuole forzare la lettura dei dati dal database senza però invalidare i dati presenti in cache. Un esempio può essere il richiamo della gestione dei dati del cliente all'interno della gestione del documento. La gestione del documento può sfruttare la cache dei dati del cliente ma nel momento in cui viene richiamata la gestione del cliente viene richiesta la riletture dei dati dal database. Questa esigenza può essere gestita con il metodo `.DisableCacheRead()`. Questo metodo è disponibile a livello di oggetto e consente di disabilitare la lettura dei dati presenti in cache per la singola istanza.

BC

```
' Disabilita la lettura da cache  
This.DisableCacheRead(1)  
This.CCF = CodiceCliFor  
This.LoadFromDB()
```

Disabilitando la lettura della cache la successiva `.LoadFromDB()` ignora i dati presenti in cache ed esegue il caricamento dati dal Database.

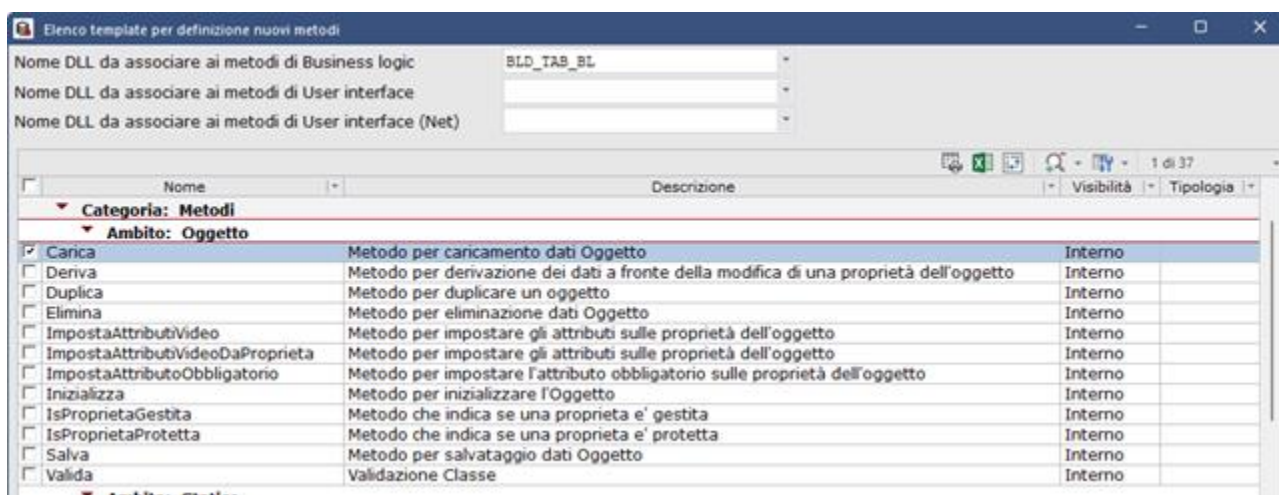
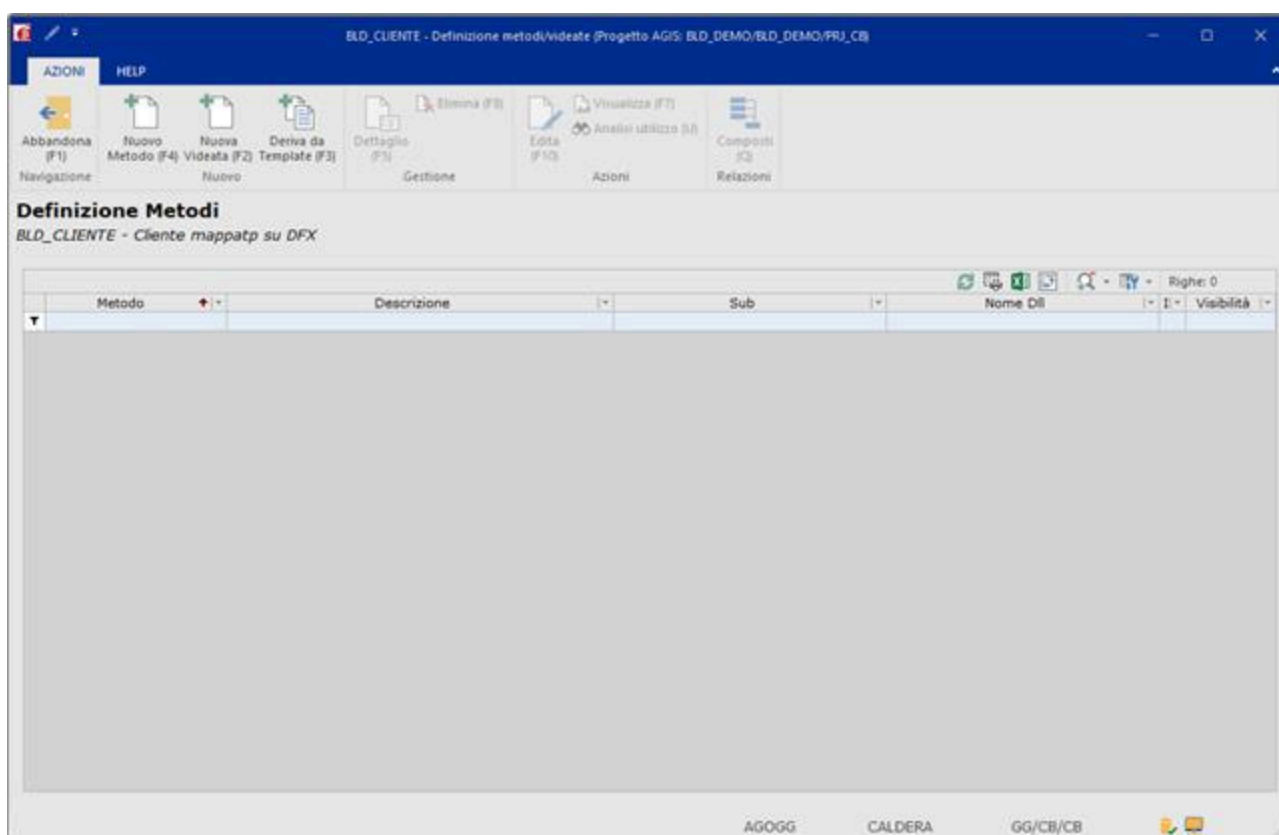
Eseguito su un oggetto composto la disabilitazione agisce su tutti i suoi componenti. Per cui anche eventuali lazy loading non utilizzeranno la cache.

I metodi

Torniamo a parlare di metodi, perché tutte le azioni di lettura, scrittura ed eliminazione sono demandate proprio a loro.

Eravamo rimasti alla possibilità di creare un metodo semplice utilizzando il bottone "Nuovo metodo". Ora possiamo andare a sofisticare un po' la tipologia di metodo da creare.

Prendiamo la classe BLD_CLIENTE mappata sul DFX BLDCLIFOR e proviamo a creare un metodo (con il bottone "Deriva da template") per la lettura dei dati da DB. Chiameremo questo metodo .Carica():



Viene presentata la videata con le informazioni del metodo da inserire:

Nuovo metodo BLD_CLIENTE

Metodo:

Descrizione:

Ambito di utilizzo:

Visibilità:

Nome sub: Nome Dll:

Sostituzione metodo base:

Firma

	Chiave	Livello	Abilita Cache	Collezione di errori	Applica profilazione	Valore di ritorno
Parametri	☒	☒	Booleano	☒	☐	Booleano

Numero di versioni previste:

OK (Invio) Abbandona (F1)

AGOGG CALDERA GG/CB/CB

Vuol dire che questo metodo verrà richiamato in automatico dall'ambiente in esecuzione al posto del metodo standard per la lettura dell'oggetto sul quale viene richiamato. Questo metodo è anche chiamato *metodo di override di lettura*.

In generale, un metodo standard può quindi andare a sostituirsi ad uno dei metodi base generati in automatico e contenuti nella DLL del contenitore di classi.

Per farlo è necessario specificare nel campo "Sostituzione metodo base" il metodo base che si intende sostituire. In questo modo, quando l'ambiente di esecuzione si trova nella situazione in cui deve eseguire il metodo base, invece di eseguire il codice autogenerato andrà a richiamare ed eseguire il codice definito dal programmatore nel metodo standard.

⚠ I metodi di override si possono creare solo su classi mappate su DFX o con la proprietà _BASE mappata su DFX.

E' necessario inoltre specificare la firma che prevede il metodo standard. Questo per fare in modo che l'ambiente possa richiamarlo in automatico specificando i parametri necessari alla chiamata della SUB.

In base alla definizione della firma scelta viene generato in automatico il sorgente relativo al metodo impostando i parametri nella '@SBP e lo sviluppo delle istruzioni minime necessarie alla sostituzione del metodo base.

Utilizzando una firma che prevede la chiave della tabella il sorgente viene generato reperendo le informazioni dal DFX di riferimento per definire in modo corretto le variabili.

⚠ I parametri devono necessariamente rispettare quanto indicato nella firma del metodo, vanno quindi mantenuti quelli proposti dalla creazione del sorgente senza ulteriori integrazioni o modifiche.

Il sorgente generato è il seguente:

BC

```
'@PG BLD_CLIENTE_CARICA STRICT[1]          ' Lettura dati da DB

'@SBP This[CLASSE[BLD_CLIENTE]]            [INOUT] _      ' Cliente
mappatp su DFX
    CFCOD[INT]                               [IN]      _      ' Codice Cliente
Fornitore
    Livello[INT]                             [IN]      _      ' Livello
massimo di caricamento dell'oggetto (-1 - Tutto; 0 - Solo il base; n - Livello di
profondità richiesto)
    UtilizzaCache[TIPO[BCBOOL]]             [IN]      _      ' Utilizza cache
se attiva
    Errori[CLASSE[BC_VID_ERROR] COLLEZIONE] [OUT]     _      ' Errori
    AS Caricato[TIPO[BCBOOL]]
```

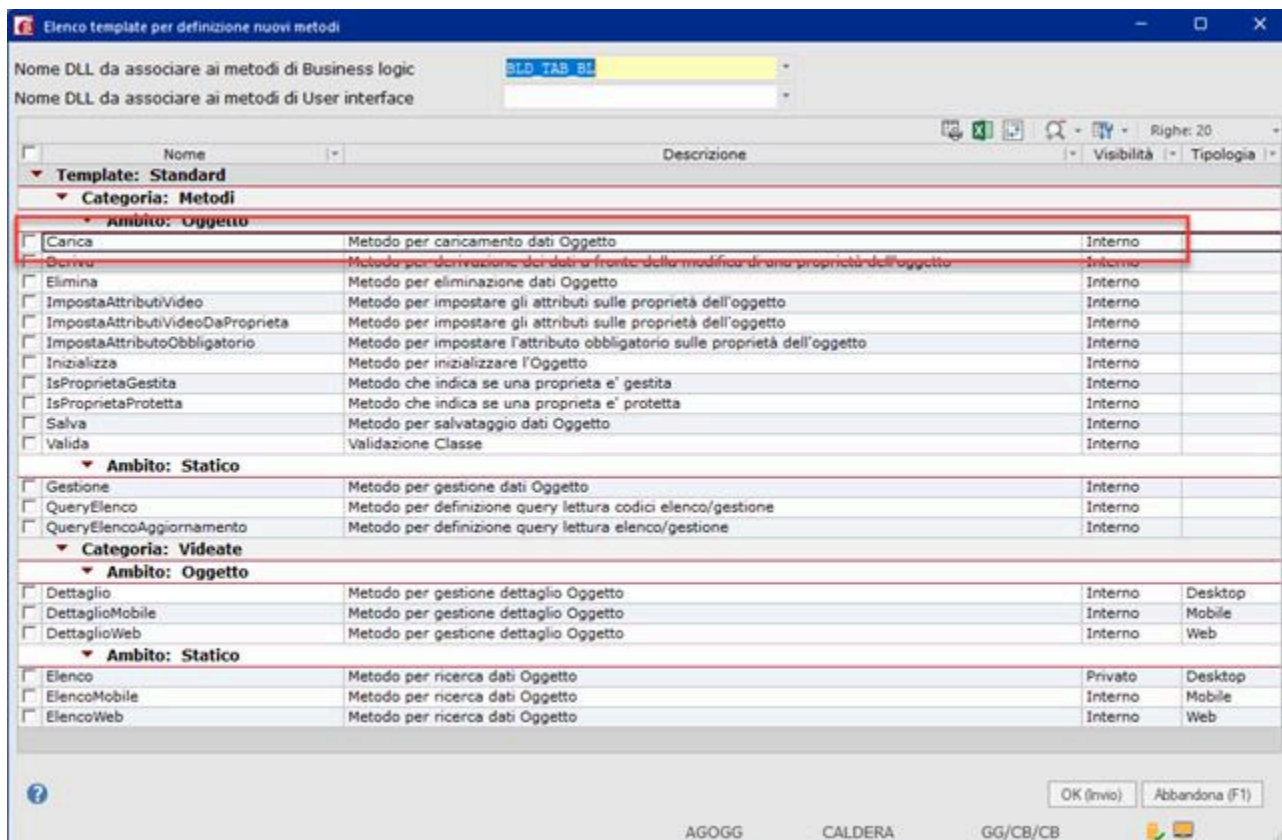
End

Vediamo che è presente il parametro CFCOD è il campo che corrisponde alla chiave principale del DFX BLDCLIFOR. Se la chiave fosse composta da più campi, sarebbero stati indicati tutti quanti.

Nel corpo del programma va indicato quanto necessario per popolare l'oggetto *This*.

Il metodo .Carica(), così come molti altri, hanno una struttura molto fissa per qualsiasi tipo di classe. L'unica cosa che cambia è l'eventuale elenco di campi che compongono la chiave. Per questo motivo sono stati generati dei template da utilizzare per creare questi metodi.

Premendo il bottone "Deriva da template" abbiamo un elenco di metodi già preconfezionati da poter creare in automatico:



L'elenco dei metodi presentati viene determinato elencando i sorgenti (template) presenti nella cartella dell'installazione di AGIS: <DATI>\TEMPLATE\METODI\CLASSI

L'elenco dei metodi Standard è guidato dal file "MetodiClassi.XML".

Si tratta di un file di definizione e di un elenco sorgenti di metodi standard, distribuiti da SISTEMI, che non possono essere personalizzati in quanto ad ogni distribuzione verranno ricoperti.



E' possibile predisporre un elenco di template per metodi personalizzati da aggiungere a quelli standard. La procedura è descritta nella scheda apposita.

Proviamo a creare il metodo sfruttando il template ed ecco come appare il nostro sorgente:

BC

22/07/26, 11:01

Lettura dei dati — Supporto Sistemi

```
'@PG BLD_CLIENTE_CARICA STRICT[1]      ' Metodo per Caricamento Oggetto

      '@SBP This[CLASSE[BLD_CLIENTE]]      [INOUT] _      ' Cliente mappatp su
DFX
      CFCOD[INT]      [IN]      _      ' Codice Cliente
Fornitore
      Livello[INT]      [IN]      _      ' Livello massimo di
caricamento dell'oggetto (-1 - Tutto; 0 - Solo il base; n - Livello di profondità
richiesto)
      UtilizzaCache[TIPO[BCBOOL]]      [IN]      _      ' Utilizza cache se
attiva
      AS Caricato[TIPO[BCBOOL]]

      This.Clear()

      ' Se non viene richiesto l'utilizzo della cache la disattiva sull'oggetto da
caricare

      If Not UtilizzaCache Then This.DisableCacheRead(1)

      This.CodCliFor=CFCOD
      This.LoadFromDb(Livello)
      If This.IsEmpty()=0 Then Caricato=#TRUE

End
```

Il corpo del programma viene popolato in automatico con la lettura da DB tramite la .LoadFromDB().

Nella tabella seguente viene specificata l'associazione tra metodo standard e metodo base e la modalità di richiamo da parte dell'ambiente.

Metodo standard	Metodo Base sostituito	Logica di richiamo
CARICA	LOADFROMMDB()	L'ambiente richiama il metodo .Carica() quando l'oggetto è componente di un composto e ne viene richiesto il caricamento dei dati. Nel caso di collezione il metodo .Carica() viene richiamato per ogni record da inserire nella collezione.
SALVA	SAVETODB()	L'ambiente richiama il metodo .Salva() quando l'oggetto è componente di un composto e viene eseguito il metodo .SaveToDB() sul composto. Nel caso di collezione il metodo .Salva() viene richiamato per ogni record da aggiornare sul database.

ELIMINA	DELFROMDB()	L'ambiente richiama il metodo .Elimina() quando l'oggetto è componente di un composto e viene eseguito il metodo .DelFromDB() sul composto. Viene inoltre chiamato nel caso in cui sia stato rimosso un oggetto dalla collezione contenuta in un oggetto composto per il quale è stato eseguito il metodo .SaveToDB(). Nel caso di collezione il metodo .Elimina() viene richiamato per ogni record da eliminare dal database.
---------	-------------	---

I metodi .Salva() ed .Elimina() li vedremo più avanti.

Visibilità dei metodi

Nella programmazione ad oggetti riveste una grande importanza la definizione dei livelli di visibilità assegnati a proprietà e metodi di una classe. Alcuni di questi concetti sono stati previsti anche in ambiente di sviluppo Sistemi per garantire il concetto di incapsulamento e di separazione di ogni oggetto software.

Livello di visibilità dei metodi

Come abbiamo visto in precedenza, i metodi si differenziano in Base e Standard. I metodi base hanno un livello di visibilità impostato d'ufficio dalla creazione della DLL contenitore. I metodi base che hanno visibilità "Privata" possono essere utilizzati soltanto nei metodi della classe per cui vengono eseguiti. Questa impostazione è un altro elemento importante per garantire a chi predispone i metodi di avere pieno controllo della logica applicativa che devono eseguire assicurandosi che non possano essere eseguite operazioni diverse da quelle necessarie.

Se ipotizziamo che il metodo carica di una classe debba eseguire altre operazioni oltre al semplice metodo base .LoadFromDB(), chi sviluppa il metodo deve avere garanzia che tutte le operazioni vadano sempre eseguite per fare in modo che la classe sia completa. Per questo motivo il metodo base .LoadFromDB() è un metodo privato e può essere utilizzato solo ed esclusivamente nelle sub di metodo della classe stessa. In questo modo, per caricare da Database i dati nell'oggetto, sarà sempre necessario richiamare un metodo standard garantendo il comportamento atteso da chi ha predisposto classe e metodi. Di conseguenza per i metodi base privati è sempre necessario creare un metodo standard, sarà poi questo ad essere utilizzato.

Diventa quindi fondamentale il concetto di sostituzione descritto nel paragrafo precedente in cui abbiamo trattato la creazione di metodi standard. Infatti per fare in modo che il metodo standard possa sostituire la gestione del rispettivo metodo base va opportunamente configurato. A questo punto, anche se richiamato in automatico dall'ambiente di esecuzione, verranno eseguite tutte le operazioni previste dal codice scritto nella SUB del metodo garantendo la completezza dell'oggetto.

A differenza dei metodi base, per i metodi standard può essere specificato il livello di visibilità da attribuire ad ogni singolo metodo. I livelli previsti sono:

- Interno
- Pubblico

Al momento il livello di visibilità dei metodi di una classe ha come unico obiettivo quello di poter evitare di esporre i metodi quando una classe viene pubblicata per essere fruita da un altro gruppo di lavoro nello sviluppo standard interno a Sistemi.

Impostando infatti il livello di visibilità a interno, nella fase di importazione di una classe in AGIS questi metodi non saranno considerati e quindi non potranno essere utilizzati.